

Project Deliverable 2

CSI 2132 - Databases I

Winter 2026

**School of Electrical Engineering and Computer Science
University of Ottawa**

Professor: Verena Kantere

Group 4

Karina Glavan 

Mia Martel 

DBMS and Programming Languages

Implementation of the Database:

For this project, we used MySQL as the database management system, along with MySQL Workbench as the IDE. The main language used for the database was SQL, which was applied to all the database-related aspects of the project.

Which includes:

- Database Implementation
- Database Population
- Database Queries
- Database Modifications
- Database Indexes
- Database Views

This section focuses solely on the database requirements of the project, the web application part is described below.

Implementation of the Web Application:

For the web application, the languages used for each side were the following:

- Client-server communication: JavaScript, HTTP, and JSON
- Server side: JavaScript using Node.js and Express
- Client side: Cascading Style Sheets (CSS), Hypertext Markup Languages (HTML), and JavaScript

Steps for Installation of Database

1. Have the database file (ehotels) open for the connection
2. Download a IDE with a terminal in powershell (recommended: Visual Studio Code)
3. Download Node.js and include the following while installing
 - Select add to PATH
 - Select npm
4. Open the folder including all of the files to run the code (Deliverable2WebApplication)
5. In the terminal, enter the following lines beside the pathways to the file (make sure your pathway goes to the Deliverable2WebApplication folder, otherwise the code will not run properly)
 - npm init -y
 - npm install express mysql2
 - node server.js (this line must be entered into the terminal every time you save changes and/or want to use the web application)
6. Enter the URL located at the top of homePage.html in a comment into your web browser and search

List of DDLs that Create the Database

Table Creation:

```
18  -- Creating the hotel chain table
19  ● CREATE TABLE Hotel_Chain (
20      ChainName VARCHAR(100) PRIMARY KEY,
21      ContactEmail VARCHAR(100) UNIQUE NOT NULL,
22      PhoneNumber VARCHAR(50) UNIQUE NOT NULL,
23      CentralOfficeAdd VARCHAR(200) NOT NULL,
24      NumOfHotels INT CHECK (NumOfHotels > 0) NOT NULL -- Constraint Added ensures each hotel chain has at least one hotel
25  );
26
27  -- Creating the hotel table
28  ● CREATE TABLE Hotel(
29      HotelID INT UNSIGNED PRIMARY KEY,
30      ContactEmail VARCHAR(100) UNIQUE NOT NULL,
31      PhoneNumber VARCHAR(50) UNIQUE NOT NULL,
32      ChainName VARCHAR(100) NOT NULL,
33      FOREIGN KEY (ChainName) REFERENCES Hotel_Chain(ChainName),
34      StarRating INT CHECK (StarRating >= 1 AND StarRating <= 5) NOT NULL,
35      NumOfRooms INT CHECK (NumOfRooms > 0 AND NumOfRooms <= 3000) NOT NULL,
36      Address VARCHAR(200) NOT NULL,
37      HotelName VARCHAR(200) NOT NULL -- Added to make it easier to import to website
38  );
39
40  -- Creating the room table
41  ● CREATE TABLE Room(
42      RoomID INT UNSIGNED PRIMARY KEY,
43      HotelID INT UNSIGNED,
44      FOREIGN KEY (HotelID) REFERENCES Hotel(HotelID),
45      CanExtend BOOLEAN NOT NULL,
46      RoomView VARCHAR(10) CHECK (RoomView = 'Mountain' OR RoomView = 'Sea' OR RoomView = 'None'), -- Changed Name of attribute from View to RoomView
47      CapacityOfRoom INT CHECK (CapacityOfRoom >= 1 AND CapacityOfRoom <= 8),
48      Price INT CHECK (Price > 0 AND Price <= 500),
49      Availability BOOLEAN NOT NULL
50  );
51
52  -- Creating a Problems_with_Room table since its considered as a multivalued attribute
53  ● CREATE TABLE Problems_with_Room(
54      RoomID INT UNSIGNED,
55      FOREIGN KEY (RoomID) REFERENCES Room(RoomID),
56      Problem VARCHAR(150) NOT NULL,
57      PRIMARY KEY (RoomID, Problem)
58  );
59
60  -- Creating an Amenity table since its considered as a multivalued attribute
61  ● CREATE TABLE Amenities(
62      RoomID INT UNSIGNED,
63      FOREIGN KEY (RoomID) REFERENCES Room(RoomID),
64      Amenity VARCHAR(200) NOT NULL,
65      PRIMARY KEY (RoomID, Amenity)
66  );
67
68  -- Creating the Customer table
69  ● CREATE TABLE Customer(
70      IDNumber INT UNSIGNED PRIMARY KEY,
71      TypeOfID VARCHAR(100) NOT NULL,
72      FullName VARCHAR(200) NOT NULL, -- Some people have long names
73      Address VARCHAR(200) NOT NULL,
74      DateOfRegistration DATE
75  );
76
77  -- Creating the Employee Table
78  ● CREATE TABLE Employee(
79      SSN_SIN INT UNSIGNED PRIMARY KEY,
80      HotelID INT UNSIGNED,
81      FOREIGN KEY (HotelID) REFERENCES Hotel(HotelID),
82      FullName VARCHAR(200) NOT NULL,
83      Address VARCHAR(200) NOT NULL,
84      PositionName VARCHAR(100) NOT NULL
85  );
```

```

87  -- Creating the Booking Table
88  CREATE TABLE Booking(
89      BookingID INT UNSIGNED PRIMARY KEY,
90      CustomerID INT UNSIGNED, -- Changed Name
91      FOREIGN KEY (CustomerID) REFERENCES Customer(IDNumber),
92      RoomID INT UNSIGNED,
93      FOREIGN KEY (RoomID) REFERENCES Room(RoomID),
94      StatusofBooking VARCHAR(20) CHECK (StatusofBooking = 'Confirmed' OR StatusofBooking = 'Pending' OR StatusofBooking = 'Cancelled') NOT NULL, -- Changed Name
95      BookingDate DATE,
96      CheckInDate DATE,
97      CheckOutDate DATE
98  );
99
100 -- Creating the Renting Table
101 CREATE TABLE Renting(
102     RentingID INT UNSIGNED PRIMARY KEY,
103     BookingID INT UNSIGNED,
104     FOREIGN KEY (BookingID) REFERENCES Booking(BookingID),
105     CustomerID INT UNSIGNED, -- Changed Name
106     FOREIGN KEY (CustomerID) REFERENCES Customer(IDNumber),
107     HotelID INT UNSIGNED,
108     FOREIGN KEY (HotelID) REFERENCES Hotel(HotelID),
109     RoomID INT UNSIGNED,
110     FOREIGN KEY (RoomID) REFERENCES Room(RoomID),
111     PaymentStatus VARCHAR(20) CHECK (PaymentStatus = 'Paid' OR PaymentStatus = 'Not Paid' OR PaymentStatus = 'Partially Paid') NOT NULL,
112     CheckInTime TIME,
113     CheckOutTime TIME
114 );

---
116 -- Adding a BookingArchive Table since was missed in the ER Diagram and Schema
117 -- So that if information about the room or customer doesnt exist the information
118 -- Exists somewhere in the database. But no Foreign Keys since the info will just be copied
119 CREATE TABLE BookingArchive(
120     BookingID INT UNSIGNED PRIMARY KEY,
121     CustomerID INT UNSIGNED, -- Changed Name
122     RoomID INT UNSIGNED,
123     StatusofBooking VARCHAR(20) CHECK (StatusofBooking = 'Confirmed' OR StatusofBooking = 'Pending' OR StatusofBooking = 'Cancelled') NOT NULL, -- Changed Name
124     BookingDate DATE,
125     CheckInDate DATE,
126     CheckOutDate DATE
127 );
128
129 -- Adding a RentingArchive Table for the same reason as BookingArchive
130 CREATE TABLE RentingArchive(
131     RentingID INT UNSIGNED PRIMARY KEY,
132     BookingID INT UNSIGNED,
133     CustomerID INT UNSIGNED, -- Changed Name
134     HotelID INT UNSIGNED,
135     RoomID INT UNSIGNED,
136     PaymentStatus VARCHAR(20) CHECK (PaymentStatus = 'Paid' OR PaymentStatus = 'Not Paid' OR PaymentStatus = 'Partially Paid') NOT NULL,
137     CheckInTime TIME,
138     CheckOutTime TIME
139 );

```

Indexes:

The following indexes were implemented to improve the performance of queries and triggers within the database:

```

1423 -- *****
1424 -- Index for Hotel Name
1425 -- *****
1426 -- EXPLANATION: Used for Query 1 for when we filter using HotelName
1427 CREATE INDEX idx_hotel_name ON Hotel (HotelName) ;
1428
1429 -- *****
1430 -- Index for Room Prices
1431 -- *****
1432 -- EXPLANATION: Used for the Second Query and the Fourth
1433 CREATE INDEX idx_room_price ON Room (Price) ;
1434
1435 -- *****
1436 -- Index for RoomID
1437 -- *****
1438 -- EXPLANATION: Used in the triggers when checking overlapping rentings and when we join renting with
1439 -- Room in some of the queries.
1440 CREATE INDEX idx_renting_room ON Renting (RoomID) ;
1441
1442 -- *****
1443 -- Index for Room Availability
1444 -- *****
1445 -- EXPLANATION: Two queries requires the filtering of available rooms, indexing would help improve the
1446 -- Performance of those queries
1447 CREATE INDEX idx_room_availability ON Room (Availability) ;
1448

```

Views:

```
1453      -- *****
1454      -- View 1: number of available rooms per area
1455      -- *****
1456
1457 • CREATE VIEW RoomsPerArea AS
1458     SELECT TRIM(SUBSTRING_INDEX(SUBSTRING_INDEX(h.address, ',', 2), ',', -1)) AS Area,
1459     COUNT(*) AS AvailableRooms FROM Hotel h
1460     JOIN Room r ON h.HotelID = r.HotelID WHERE r.Availability = TRUE
1461     GROUP BY Area;
1462
1463      -- *****
1464      -- View 2: aggregated capacity of all the rooms of a specific hotel
1465      -- *****
1466
1467 • CREATE VIEW roomCapacity AS
1468     SELECT h.HotelName, SUM(r.CapacityOfRoom) AS TotalCapacity FROM Hotel h
1469     JOIN Room r ON h.HotelID = r.HotelID
1470     GROUP BY h.HotelID, h.HotelName;
```

Triggers:

```
1261      -- *****
1262      -- Trigger for when a renting is created so that room turns unavailable
1263      -- *****
1264      DELIMITER //
1265 • CREATE TRIGGER update_room_availability_after_renting
1266     AFTER INSERT ON Renting
1267     FOR EACH ROW
1268     BEGIN
1269         UPDATE Room
1270         SET Availability = FALSE
1271         WHERE Room.RoomID = NEW.RoomID ; --
1272     END //
1273     DELIMITER ;
1274
1275      -- *****
1276      -- This trigger ensures there are no overlapping rentings with date and time
1277      -- *****
1278
1279      DELIMITER //
1280 • CREATE TRIGGER prevent_overlapping_rentings
1281     BEFORE INSERT ON Renting
1282     FOR EACH ROW
1283     BEGIN
1284         IF EXISTS (
1285             SELECT 1
1286             FROM Renting
1287             JOIN Booking b1 ON Renting.BookingID = b1.BookingID
1288             JOIN Booking b2 ON b2.BookingID = NEW.BookingID -- where b1 is an existing booking (which connects to renting) and b2 the new booking being converted to renting
1289             WHERE Renting.RoomID = NEW.RoomID
1290             AND (
1291                 b2.CheckInDate < b1.CheckOutDate
1292                 AND b2.CheckOutDate > b1.CheckInDate
1293             )
1294         ) THEN
1295             SIGNAL SQLSTATE '45000'
1296             SET MESSAGE_TEXT = 'Error: Rentings Overlap';
1297         END IF;
1298     END //
1299     DELIMITER ;
1300
```

```

1301 -- *****
1302 -- Trigger for archiving bookings and rentings for if a booking/renting is deleted
1303 -- *****
1304
1305 -- Booking archive
1306 DELIMITER //
1307 • CREATE TRIGGER archive_booking_before_delete
1308 BEFORE DELETE ON Booking
1309 FOR EACH ROW
1310 BEGIN
1311     INSERT INTO BookingArchive
1312     VALUES (
1313         OLD.BookingID,
1314         OLD.CustomerID,
1315         OLD.RoomID,
1316         OLD.StatusofBooking,
1317         OLD.BookingDate,
1318         OLD.CheckInDate,
1319         OLD.CheckOutDate
1320     );
1321 END //
1322 DELIMITER ;
1323

```

```

1324 -- Renting Archive
1325 DELIMITER //
1326 • CREATE TRIGGER archive_renting_before_delete
1327 BEFORE DELETE ON Renting
1328 FOR EACH ROW
1329 BEGIN
1330     INSERT INTO RentingArchive
1331     VALUES (
1332         OLD.RentingID,
1333         OLD.BookingID,
1334         OLD.CustomerID,
1335         OLD.HotelID,
1336         OLD.RoomID,
1337         OLD.PaymentStatus,
1338         OLD.CheckInTime,
1339         OLD.CheckOutTime
1340     );
1341 END //
1342 DELIMITER ;
1343
1344 -- Adding a new trigger that will make the room available after renting is deleted
1345 -- Typically done after they check out, so that the room always stays available
1346 DELIMITER //
1347 • CREATE TRIGGER restore_availability
1348 AFTER DELETE ON Renting
1349 FOR EACH ROW
1350 BEGIN
1351     UPDATE Room
1352     SET Availability = TRUE
1353     WHERE RoomID = OLD.RoomID;
1354 END //
1355 DELIMITER ;
1356

```